

Author 1:	Jonathan Mabery
Email 1:	jmabery@uco.edu
Author 2:	Andrew Aprile
Email 2:	aaprile@uco.edu
Course:	ADS. CMSC3613. CRN10040
Assignment	Project 6. P06

Give a brief discussion of your implementation: just like an explanation of your idea in an interview.

To calculate optimal paths based on cost or time, we used a “Dual Graph Separation” strategy. The input parser iterates through the raw adjacency matrix, splitting both cost and time using a pipe ‘|’ character as a delimiter. (e.g Time|Cost). The design simplified query processing by using two Digraph objects, which allowed the program to simply direct queries to either the graphTime or graphCost instance based on the user's mode selection (T or C).

Another discussion: in this implementation, the graph is represented by an adjacency matrix, and the vertex is in int type. But what if (1) we want to design a new class type for the Vertex to include the name of the city and (2) we want to design a new class type for the Weight to include both the time and cost, then how would you like to represent the graph? Please briefly explain your ideas, and no real code is required.

Instead of using an adjacency matrix, we would probably want to use an adjacency list, implemented through a Hash Map. (for example, Map<City,List<Edge>>) In this implementation, the City object acts as the key, which allows direct lookups without an integer mapping table. This is all while the value is a list of Edge objects that contain the destination City and a FlightData object that has both Time and Cost variables.

Screenshots of a test run.

The screenshot shows three windows from a code editor:

- input2.dat:** Contains the number 4, a list of cities (Austin Chicago Dallas Houston), and an adjacency matrix with time and cost values separated by pipes.
- query2.d:** Contains two queries: "Dallas Austin T" and "Chicago Dallas C".
- output2.:** Contains the results: "No available path" for the first query and "289" for the second query.

Below the windows, a terminal window shows the command and output:

```
$ ./p06 input2.dat query2.dat output2.dat

Dallas to Austin by Time: No available path
Chicago to Dallas by Cost: $289
```

input3.dat	query3.dat	output3.dat
<pre>4 Austin Chicago Dallas Houston 0 0 INF INF 3 47 2 39 4 200 0 0 8 50 9 50 INF INF 5 192 0 0 18 51 7 1 1 50 INF INF 0 0</pre>	<pre>Austin Chicago C Austin Chicago T Dallas Houston T</pre>	<pre>89 3 11</pre>

```
$ ./p06 input3.dat query3.dat output3.dat
```

```
Austin to Chicago by Cost: $89
Austin to Chicago by Time: 3 minutes
Dallas to Houston by Time: 11 minutes
```